1. Why Do We Need Architecture in Any Project?

Software architecture is the high-level structure of an application. It defines the major parts, their responsibilities, and how they communicate. It gives the team a plan for organizing code as the system grows.

- Separation of concerns: UI, business rules, and data access do not become mixed together.
- Maintainability: changes in one area are less likely to break unrelated areas.
- Scalability: the application can evolve as users, data, and features increase.
- Testability: business logic can be tested without starting the whole application or requiring a real database.
- Team collaboration: clear boundaries let developers work on different modules.
- Reusability: common services and rules can be reused.
- Reduced complexity: dependencies and data flow are easier to understand.

Example

In a college management system, a Student screen should not contain SQL queries and all business rules. The UI calls an application/service layer; that layer applies rules and uses a data-access component to communicate with the database.

2. What Is N-Tier Architecture?

N-Tier Architecture divides an application into logical or physical tiers. Each tier has a defined responsibility and communicates with other tiers through agreed interfaces.

Presentation Tier	Business Logic Tier	Data Access Tier
UI: MVC Views, Razor Pages, Web API endpoints. Receives input and displays output.	Rules and use cases: validation, calculations, workflows, and application decisions.	Database communication: EF Core, repositories, queries, and persistence.

Typical flow: Presentation → Business Logic → Data Access → Database.

Advantages

- Clear separation of responsibilities.
- Each tier can often be changed and tested more independently.
- Supports team organization and reuse.
- Tiers can be deployed on separate machines when physical separation is needed.

Trade-offs

- Extra layers can add boilerplate and complexity.
- Poorly designed layers may only pass data through.
- Physical separation adds network latency and operational cost.

Layer vs. Tier

A layer is a logical responsibility boundary; a tier often refers to a physical deployment boundary. The terms are sometimes used interchangeably, but they are not exactly the same.

3. What Is Onion Architecture?

Onion Architecture puts the domain model and business rules at the center. Infrastructure details—such as databases, external services, and frameworks—are in outer layers. Dependencies point inward, so the core does not depend on infrastructure.

- Domain (center): entities, value objects, domain rules, and domain interfaces.
- Application: use cases, application services, commands/queries, DTOs, and required abstractions.
- Infrastructure: implementations for persistence, EF Core, file storage, email, and external APIs.
- Presentation: MVC, Web API, UI, or other entry points that invoke application use cases.

Dependency rule

Outer layers may depend on inner layers. Inner layers must not depend on outer layers. For example, Application defines an IStudentRepository interface; Infrastructure implements it using EF Core. Dependency Injection connects them at runtime.

```
// Application layer
public interface IStudentRepository
{
    Task<Student?> GetByIdAsync(int id);
}

// Infrastructure layer implements the interface
public class StudentRepository : IStudentRepository
{
    private readonly AppDbContext _context;
    public StudentRepository(AppDbContext context) => _context = context;

    public Task<Student?> GetByIdAsync(int id) =>
        _context.Students.FindAsync(id).AsTask();
}
```

Benefits and comparison

Onion keeps business rules independent of database and UI technologies, improves unit testing, and makes infrastructure replaceable. N-Tier describes responsibility layers and a common top-to-bottom flow; Onion emphasizes dependency direction. A project can use both ideas.

4. Is LINQ Slow? Deferred vs. Eager Execution

LINQ is not automatically slow. Performance depends on the query, data source, amount of data, generated SQL (with EF Core), indexes, and when the query executes.

Deferred execution

Many LINQ operators, such as Where and Select, build a query but do not execute it immediately. Execution occurs when the sequence is enumerated, for example by foreach, ToList(), or ToArray().

```
var query = numbers.Where(n => n > 10); // query is defined

foreach (var number in query)              // execution occurs here
{
    Console.WriteLine(number);
}
```

Deferred execution can avoid work until needed and supports query composition. However, enumerating the same query more than once may execute it more than once.

Eager / immediate execution

Operators such as ToList(), ToArray(), Count(), First(), FirstOrDefault(), and Sum() execute immediately and return a materialized or calculated result.

```
var students = await context.Students
    .Where(s => s.Age >= 18)
    .ToListAsync(); // EF Core executes the query now
```

With EF Core, Where and Select are generally translated into SQL. ToListAsync() sends the query and materializes matching rows in memory.

Why LINQ can appear slow

- Loading too many rows instead of filtering in the database.
- Calling ToList() too early and filtering in memory.
- Executing the same deferred query repeatedly.
- Inefficient SQL or missing database indexes.
- N+1 query patterns when loading related entities.
- Expensive work inside projections or loops.

5. Three Use Cases for Asynchronous Programming

Async programming is useful when an operation takes time and the current thread would otherwise be blocked waiting. In .NET, async and await make non-blocking code easier to write.

Use case 1: External HTTP/API calls

A web app may call a payment, shipping, identity, or weather API. Network responses can take time, so use asynchronous I/O.

```
public async Task<string> GetDataAsync(HttpClient client)
{
    return await client.GetStringAsync("https://example.com/data");
}
```

Use case 2: Database operations

Database queries and saves involve I/O. EF Core provides ToListAsync, FirstOrDefaultAsync, and SaveChangesAsync.

```
public async Task<List<Student>> GetStudentsAsync()
{
    return await _context.Students
        .Where(s => s.Age >= 18)
        .ToListAsync();
}
```

Use case 3: File and stream operations

File uploads, downloads, and stream operations may wait on storage or a network.

```
string content = await File.ReadAllTextAsync("notes.txt");
await File.WriteAllTextAsync("copy.txt", content);
```

Important note

Async does not automatically make CPU-heavy work faster. Parallelism or Task.Run may be appropriate for some CPU-bound scenarios. In ASP.NET Core, avoid wrapping normal I/O calls in Task.Run; use the API's native async method.

6. What Is the Difference Between a Thread and a Task?

Aspect	Thread	Task
Meaning	An operating-system execution thread.	A .NET abstraction representing an operation and its completion.
Level	Lower-level execution resource.	Higher-level programming abstraction.
Scheduling	Can be created and managed directly.	Usually scheduled by the Task Scheduler; CPU work commonly uses the ThreadPool.
Result	No built-in result model.	Task<TResult> represents an operation that returns TResult.
Composition	Coordination is more manual.	Supports await, WhenAll, WhenAny, continuations, and cancellation patterns.
Async I/O	A thread need not remain blocked while I/O is pending.	A Task can represent pending I/O without dedicating a thread to the wait.

```
// Direct thread creation
var thread = new Thread(() => Console.WriteLine("Dedicated thread"));
thread.Start();

// Task-based CPU work
Task task = Task.Run(() => Console.WriteLine("ThreadPool work"));
await task;
```

A Task is not a thread. It may run on a ThreadPool thread, or represent asynchronous I/O that does not occupy a thread while waiting. A Task represents work and completion; a Thread is an execution resource.

7. What Is New in .NET 9 for Operations on Generic Types?

This wording can refer to two related topics. The important interview distinction: Generic Math was introduced before .NET 9, while .NET 9 ships with C# 13 and adds a generic capability for ref struct types.

A. Generic Math — .NET 7 / C# 11

Generic Math uses static abstract members in interfaces such as INumber<TSelf>. It allows one generic method to perform numeric operations on supported types without writing separate overloads.

```
using System.Numerics;

static T Add<T>(T first, T second)
    where T : INumber<T>
{
    return first + second;
}

Console.WriteLine(Add(10, 20));   // int
Console.WriteLine(Add(2.5, 3.5)); // double
```

Do not say Generic Math was introduced in .NET 9. It arrived in .NET 7 with C# 11.

B. C# 13 / .NET 9: allows ref struct

C# 13, shipped with the .NET 9 SDK, introduces the allows ref struct anti-constraint. It allows a generic type parameter to accept a ref struct type, subject to ref-safety rules. This is useful for types such as Span<T> and ReadOnlySpan<T>.

```
public class BufferProcessor<T>
    where T : allows ref struct
```

```
{
    public void Process(scoped T buffer)
    {
        // Use T while following ref-safety rules.
    }
}

// Example generic argument:
var processor = new BufferProcessor<Span<int>>();
```

The exact use is subject to compiler and ref-safety restrictions; this snippet illustrates the generic constraint.

Other related C# 13 changes

- ref struct types can implement interfaces, with restrictions.
- ref locals and unsafe contexts are allowed in more async and iterator scenarios, but ref-like values cannot cross await or yield boundaries.

8. What Are the Built-in Delegate Types in C#?

A delegate is a type-safe reference to a method. A delegate variable can store a method with a compatible parameter list and return type. Delegates are used for callbacks, events, and passing behavior as an argument.

1. Action

Action represents a method that returns void. It supports zero or more input parameters.

```
Action<string> printName = name => Console.WriteLine(name);
printName("Mahmoud");
```

Examples: Action, Action<int>, Action<string, int>. Generic arguments represent input parameter types; there is no return type.

2. Func

Func represents a method that returns a value. The last generic type argument is the return type; preceding arguments are input types.

```
Func<int, int, int> add = (a, b) => a + b;
int result = add(5, 3); // 8
```

Examples: Func<int> (no inputs, returns int), Func<int, string> (takes int, returns string), Func<int, int, bool> (takes two ints, returns bool).

3. Predicate<T>

Predicate<T> represents a method that takes one T value and returns bool. It is often used to test a condition.

```
Predicate<int> isEven = number => number % 2 == 0;
Console.WriteLine(isEven(8)); // True
```

4. EventHandler and EventHandler<TEventArgs>

These are common delegate types in the .NET event pattern. EventHandler is used without custom event data; EventHandler<TEventArgs> carries event data.

```
public class Order
{
    public event EventHandler? Shipped;

    public void Ship()
    {
        Shipped?.Invoke(this, EventArgs.Empty);
```

```
    }
}

// Subscribe:
order.Shipped += (sender, args) => Console.WriteLine("Order shipped!");
```

Quick comparison

Type	Inputs	Return	Typical use
Action<T...>	Zero or more	void	Callback / execute operation
Func<T..., TResult>	Zero or more	TResult	Calculate and return a value
Predicate<T>	One T	bool	Test a condition
EventHandler<TEventArgs>	sender + event args	void	Events

Custom delegate

You can declare your own delegate when a named signature improves clarity or is required by an API.

```
public delegate decimal DiscountCalculator(decimal price);

DiscountCalculator applyDiscount = price => price * 0.90m;
Console.WriteLine(applyDiscount(100m)); // 90
```

-